Air Traffic Disruption Thesis

Anmol Sapru

2026-02-12

Table of contents

1 Why Care? **2**

2 Data **3**
2.1 General Data Cleaning 4

3 Hub-and-Spoke Carriers **8**

4 Point-to-Point Carriers **11**

5 Network Simulation **12**
5.1 Airport Classification 12
5.2 Delay Generation Functions 12
5.3 Weather Scenario Generator 13
5.4 Airport Queue Manager 13
5.5 Flight State Machine 14
5.6 Core Simulation Engine 14
5.7 Passenger Connection Analysis 14
5.8 Monte Carlo Framework 14
5.9 Validation Framework 14
5.10 Sensitivity Analysis 14
5.11 Visualization Functions 14

6 Running the Simulation **14**
6.1 Key Improvements Over Original Simulation 14

7 References **14**

1 Why Care?

The United States' air transportation system is one of the most complex networks in the world. On any given day, approximately 45,000 flights operate across the country, carrying roughly 2 million passengers through a web of airports, airways, and airline operations that are deeply interdependent. This interconnectedness is an achievement of modern logistics but also fundamental vulnerability: when one node in the network falters, the effects rarely stay contained–which means the impact of even small disturbances can be widespread.

In operations research, this tension is well understood. Systems optimized for operational efficiency, with tight turnaround times, high aircraft utilization, at the mercy of mechanical, crew limits, infrastructure constraints for the expense of robustness. The result is a network that performs exceptionally well on a clear day but degrades rapidly when disruptions occur. A single delayed aircraft can trigger a chain reaction: its next flight departs late, connecting passengers miss their transfers, gate assignments shift, crew duty limits are exceeded, and the downstream consequences multiply across airports and airlines. What begins as a localized problem at one gate can, within hours, become a system-wide event affecting thousands of passengers at dozens of airports.

This project aims to build a simulation that can model such disruptions at scale, approaching the problem from the perspectives of robustness, recovery, and systemic impact. Rather than examining individual flights or isolated airports, the simulation will capture the network-level dynamics that make air transportation disruptions so difficult to manage—how delays propagate, how different actors respond, and how those responses interact to either contain or amplify the original disturbance.

A central goal of the simulation is to understand the role that airport hubs play in either mitigating or exacerbating disruption cascades. Hub airports sit at the center of airline networks and handle a disproportionate share of connecting traffic. Their performance during disruptions—how quickly they recover, how effectively they manage passenger reaccommodation, how their operational decisions affect downstream airports—can determine whether a localized weather event remains a regional inconvenience or becomes a national crisis. By modeling these dynamics, the simulation aims to identify what characteristics of a hub airport contribute to network resilience and how hub selection and design might be improved to reduce systemic vulnerability.

Air traffic disruptions have been studied through a variety of analytical lenses. Network theory has been used to characterize the structural properties of the air transportation system—identifying critical nodes, measuring connectivity, and assessing vulnerability to targeted failures. Agent-based modeling has provided frameworks for representing the decentralized decision-making that occurs during disruptions, where airlines, airports, air traffic control, and passengers each pursue their own objectives under incomplete information. Operational research methods have been applied to optimize recovery strategies, including aircraft rerouting, crew reassignment, and passenger rebooking.

These approaches have collectively demonstrated that disruptions can cascade rapidly through the network, that the decisions of different actors are deeply interdependent, and that the system's response to a disruption is often nonlinear—small perturbations can sometimes be absorbed, while seemingly similar perturbations under slightly different conditions can trigger widespread failures. However, most existing models address only one dimension of the problem: they focus on airline operations without modeling airport constraints, or they capture network structure without representing the behavioral dynamics of the agents involved.

2 Data

Finding the proper data required going down multiple paths, since I knew that no one dataset would be perfect for what I needed. This included examining Global Distribution System data (cost-prohibitive, but would require near-zero cleaning) and data from the Department of Transportation (no data that gave information transformable to departure and arrivals), among others.

As I looked through the trade-offs of the data, I decided to sacrifice some perfection in the scheduling information to get more reproducibility and a lowered cost. Removal of the proprietary data sets would make the results of the projects able to be easily tried over other time periods or region.

Therefore, the work incorporated OpenSky data, which provides access to live and historical flight information. This allowed the simulation to reflect realistic aircraft positioning, which was used to create departure and arrival schedules. By leveraging their Trino database, I was able to get historical flight data, utilising a pull from October 6, 2025 and October 27, 2025. This was a relatively calm period for aviation in the contiguous United States, with few weather events and avoiding the Federal Aviation Administration's Air Traffic Control shutdown, giving me a clean slate.

A balance was made between the hub-and-spoke and point-to-point flying airlines of the United States. The cutoff used were airlines with at least 100 airplanes in their fleets: Alaska Airlines, American Airlines, Delta Air Lines, United Airlines, Frontier Airlines, JetBlue, Southwest Airlines, and Spirit Airlines. Some of these mainline airlines also have regional carriers that fly, which were pulled from OpenSky too.

The core dataset (all_data) contains 455,672 flight records across the October 6–27, 2025 study period. Each row represents a single observed flight segment, with the following fields:

- icao24 is the unique 24-bit transponder address assigned to each aircraft, which serves as the primary identifier for tracking individual airframes across multiple flights and days
- callsign contains the airline flight number as broadcast by the aircraft's transponder, which links the observed flight to a specific scheduled service.

- firstseen and lastseen record the UTC timestamps when OpenSky's ADS-B receiver network first and last detected the aircraft's transponder signal for a given flight segment
- departure and arrival contain the ICAO airport codes inferred by OpenSky based on the aircraft's position at the start and end of each segment

A key problem that existed with this data was a spike around midnight, where the flights that left before 11 PM EST but did not land before 12 PM EST were erroneous given a departure time of midnight the next day. This shortened the flight time and required some changes to the scheduling for point-to-point flights.

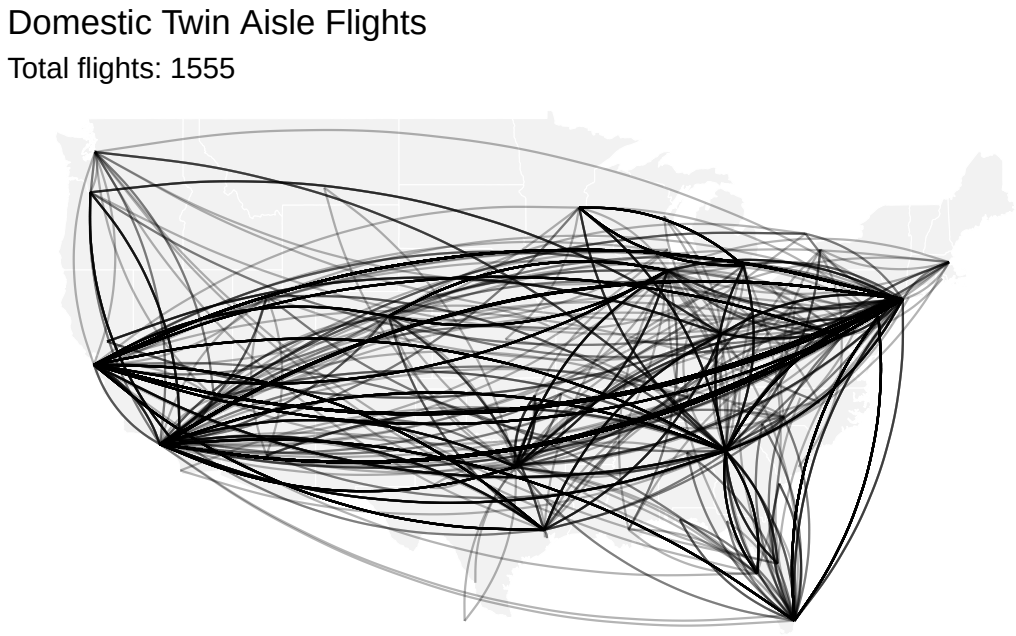
2.1 General Data Cleaning

After pulling from OpenSky, I needed to do some cleaning of the data. First was making sure the airports were correct. The only airports used were those with ICAO codes that began with 'K', which represents the contiguous United States. Since OpenSky uses the transponder location instead of the schedule data there were some assumed departure and arrival airports that did not make much sense. Some replacement of airports needed to be done and I only used airports from OurAirports that were sized as large or medium with scheduled service—after checking the small airports and determining to not be relevant airports for scheduled service of the airlines.

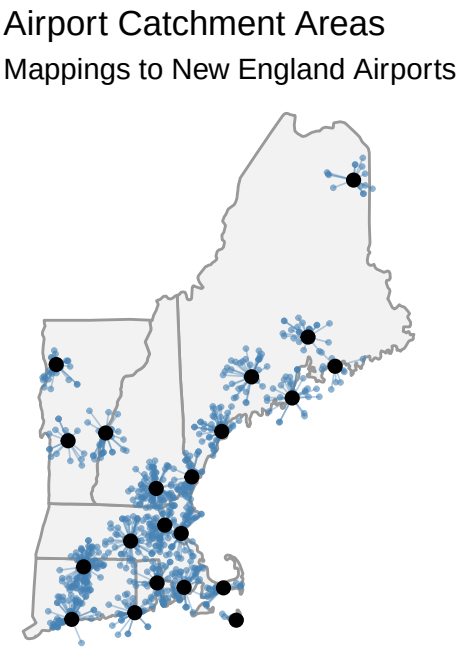
Scheduled Service Airports (US Lower 48)

I removed twin aisle jets from the data of airplanes, seeing as most of them do travel outside the contiguous United States and the analysis was focused on this area. OpenSky supplies a

Complete Aircraft Database that allowed for checking the hexadecimal codes for airplanes to their ownership and type. For other planes not in this data set, a check of if they completed a flight during the period which was over 3500 nautical miles (the current longest single aisle jet flight operated by US airlines—EWR to ARN by United Airlines is just under this threshold).



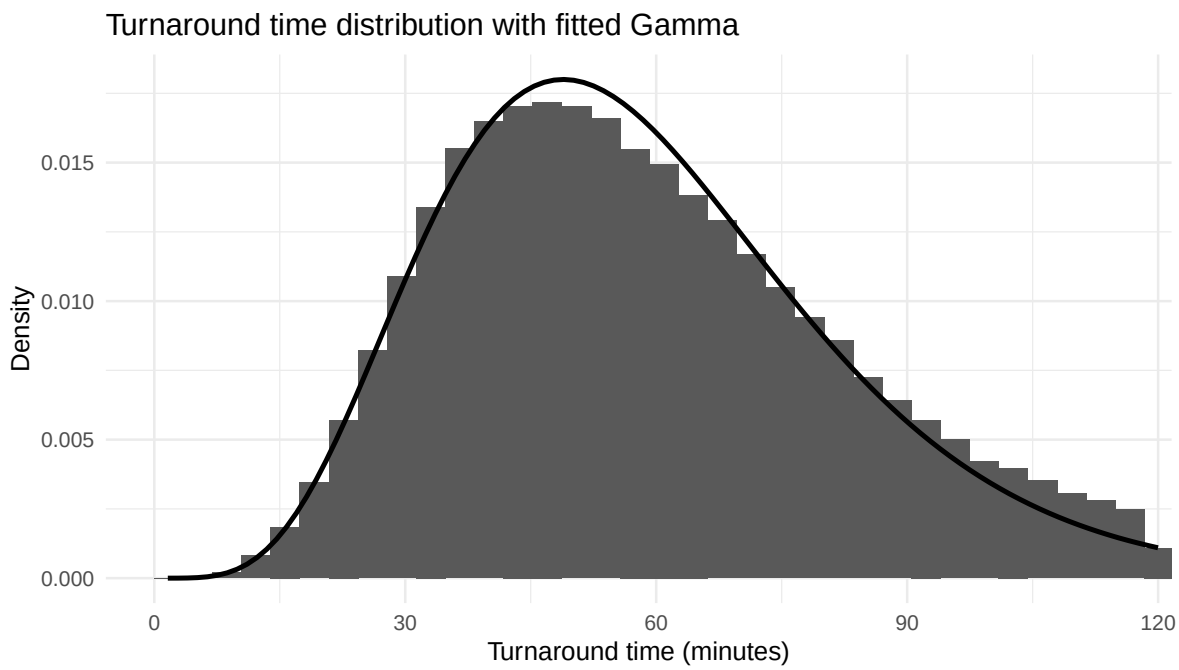
The replacement was done with a simple closest airport calculation for the non-OurAirports ICAOs. In addition to using the OurAirports, the Federal Aviation Administration's Airport Master Record has location about some of the airfields. The longitude and latitudes were calculated to those in OurAirports, and then setting a limit to within 40 kilometers—with anything further being removed due to either being too uncertain or potentially an international flight that was mistakenly categorized as flying domestically. The longitude and latitude for the commercial airports was gotten through the airportr package.



Other data cleaning involved lots of work with the departure and arrival timezones. First, a reference table of the airports from the airportr package was built, which included timezone was determined by passing its latitude and longitude through the lutz package. With valid timezones in place, the UTC-stamped firstseen and lastseen columns could be converted to local departure and arrival times.

```
Gamma shape: 6.031405
Gamma rate: 0.1028985
```

The turnaround time of an airplane was modeled as a Gamma distribution, which makes sense as its definition of "waiting time". For each aircraft, identified by its unique icao24 transponder code, flights were sorted chronologically and the gap between consecutive flights was calculated as the difference between one departure and its previous arrival. These gaps were filtered to exclude negative values and those exceeding two hours, focusing on ground turnaround times instead of overnight stays or maintenance stops.



3 Hub-and-Spoke Carriers

For hub and spoke airlines, some key ideas are the hubs they have, the banks they operate at airports, and destinations flown to.

Table 1: Top 12 hubs by Departure Volume

United (UAL)	Delta (DAL)	American (AAL)	Alaska (ASA)
KDFW	KSEA	KATL	KORD
KCLT	KPDX	KDTW	KDEN
KORD	KSAN	KMSP	KIAH
KMIA	KSFO	KSLC	KEWR
KPHX	PANC	KLAX	KSFO
KPHL	KLAX	KLGA	KIAD
KDCA	KPHX	KSEA	KLAX
KLAX	KLAS	KBOS	KLAS
KMCO	KSNA	KJFK	KMCO
KBOS	KGEG	KMCO	KBOS
KLGA	KJFK	KLAS	KSAN
KLAS	KORD	KRDU	KAUS

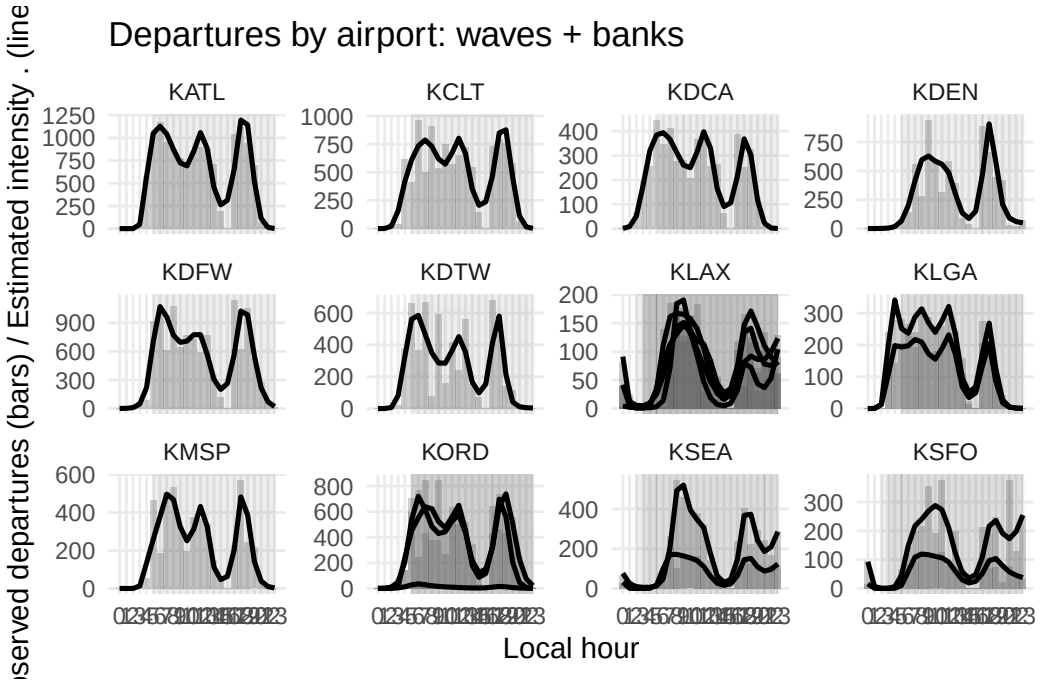
The top 12 airports for each airlines were selected as hubs. For airlines (such as United) that did not have this many airports, the future results of the code were not affected by this overestimate, since flights between hubs were scheduled in the same way as those from a hub to spoke.

The regional airlines are a huge part of the networks that are being flown. Some of the regional airlines only fly for a single larger counterpart (Endevour for Delta or Horizon for Alaska), but the larger Republic and Skywest fly for multiple airlines. Therefore, the number of hubs that each airplane takes-off or lands at was used to determine what marketable airline each one belonged to.

To understand the banking at different hubs, a GAM with a cyclic cubic spline is fit to hourly flight counts, which gives a smooth Poisson rate curve across the day. This curve is then passed to an algorithm that identifies the peaks and throughs The deepest valleys serve as cut points that partition the operating day (defined as 5:00 to 23:00 local time) into departure banks, allowing to mimick the wave structure characteristic of hub-and-spoke scheduling.

Within each bank, departure slots are optimized using a connectivity scoring function. For a candidate departure time, the score counts how many arriving flights fall within a feasible connection window—after the minimum connect time of 35 minutes but before the maximum of 180 minutes—giving preference to departure times that maximize the number of passengers who can connect from inbound flights. The optimizer initializes slots at evenly spaced positions within each bank and then iteratively shifts each slot within a local search radius to maximize its connectivity score, respecting minimum separation constraints between adjacent slots.

Destinations are then allocated to slots within each bank using a proportional assignment method. Each destination receives a number of slots proportional to its observed frequency, with fractional remainders resolved by a largest-remainder rounding scheme. The resulting assignments are shuffled randomly to avoid systematic ordering effects. Finally, utility functions handle flight duration lookup from historical medians and coordinate enrichment by joining departure and arrival airport positions for downstream mapping.




```
Starting 7-day hub-spoke simulation...

Simulating: Monday
Simulating: Tuesday
Simulating: Wednesday
Simulating: Thursday
Simulating: Friday
Simulating: Saturday
Simulating: Sunday

Hub-spoke schedule saved: 89062 flights
```

The creation of schedule for hub-and-spoke carriers first estimates demand by calculating how many departures each airline needs per bank at each hub on each day of the week, using the 90th percentile of observed daily counts as the target to size the schedule for a busy-but-not-extreme day. It then computes route probability shares for each destination within each bank, downweighting routes to the airline's own hubs by 50% to avoid over-saturating hub-to-hub connections relative to spoke destinations. Alongside this, it builds a minute-resolution vector of average arriving flights at each hub, which the connectivity optimizer later uses to score candidate departure times.

The daily schedule generator brings these inputs together: for a given day, it optimizes departure slot times within each bank to maximize connectivity with inbound flights, assigns

destinations to those slots proportionally, looks up historical block times for each route, and generates matching return flights assuming a 45-minute turnaround at the spoke. This generator is then called within a seven-day fleet simulation loop that assigns tail numbers using a greedy algorithm. For each flight, the simulator checks whether an available aircraft of the right airline is already at the origin airport and reuses it if so, otherwise introducing a new tail number into the fleet. Aircraft availability carries over between days, allowing the simulation to track how many unique aircraft each airline actually needs across the full week.

4 Point-to-Point Carriers

For hub and spoke airlines, some key ideas are the routes they operate

The previously discussed midnight spike was artificially dealt with here through comparing the flight count in hour zero against the median of hours one through three for each airline and date. Any excess above that expected baseline is identified, and a random sample of that many flights is selected from the midnight hour and shifted backward by one or two hours (with a bias toward smaller shifts).

```
Simulating FFT with 161 planes...
Simulating JBU with 268 planes...
Simulating NKS with 150 planes...
Simulating SWA with 801 planes...
```

```
Point-to-point schedule saved: 66216 flights
```

Each aircraft for point-to-point carriers is treated as an independent agent traversing a Markov chain of routes. First, it extracts fleet sizes from the observed data by counting unique transponder codes per airline, computes median block times for every route, and builds a transition matrix capturing the probability of flying to each destination given a current airport. Each simulated aircraft starts at a randomly chosen airport weighted by observed departure frequency, then repeatedly samples its next destination from the transition matrix, flies the median block time, turns around in 45 minutes, and continues until the seven-day simulation window expires.

```
Combined schedule: 155278 flights
```

5 Network Simulation

The simulation models a seven-day operating week at one-minute resolution.

Airport capacity is expressed as the maximum combined number of arrivals and departures per hour, tiered by FAA hub classification. The airports classified as Large Hub have 90 operations per hour (e.g. ATL, LAX), Medium Hub have 60 operations per hour (e.g. CLT, IAH), Small Hub's have 40 operations per hour (e.g. BNA, RDU), and Smaller non-hub regional airports have 20 operations per hour.

Turnaround times are drawn from Gamma distributions whose shape and rate parameters are fitted to observed data earlier. These could be further expanded by airport-airline pairs, but at the moment the amount of data is not large enough to support such update.

The simulation models three exogenous delay sources, each with an independent probability of occurrence and a parametric severity distribution. The block times are generated by multiplying the scheduled block time by a normally distributed factor (mean 1.0, standard deviation 0.08), bounded between 0.8 and 1.3. This reflects the natural variability in taxi times, en-route winds, and ATC routing.

Passenger loads are drawn from a normal distribution centered on 85% of an assumed 150-seat average capacity. Thirty percent of passengers are designated as connecting, enabling downstream misconnection analysis.

5.1 Airport Classification

5.2 Delay Generation Functions

Each delay source is encapsulated in a dedicated generator function that accepts a count of flights and returns a vector of delay values in minutes.

The carrier delay is determines whether each flight is affected (probability 0.08). Affected flights receive a delay sampled from a log-normal distribution (meanlog 2.5, sdlog 0.8), capped at 300 minutes. The log-normal family captures the heavy right tail observed in maintenance and crew delays.

Similarly gated by a Bernoulli probability of 0.05, air traffic control delays follow a Gamma distribution (shape 2.0, rate 0.1) capped at 180 minutes. The Gamma shape produces a moderate right skew consistent with ground delay programs and airspace flow restrictions.

Weather delays use an adjusted base probability multiplied by an airport-specific weather factor that intensifies during active weather events. Severity follows a Gamma distribution (shape 1.5, rate 0.05) with a generous 360-minute cap, reflecting the potentially extreme duration of convective weather holds.

5.3 Weather Scenario Generator

The weather subsystem enables analysts to inject localized disruption events into the simulation. Each weather event is defined by a type, a set of affected airports, a time window, and an intensity modifier.

Scenario Capacity Factor Delay Factor Ground Stop Clear 1.0 1.0 No Thunderstorms 0.5 4.0× No Winter Storm 0.3 6.0× No Ground Stop 0.0 10.0× Yes Low Visibility 0.6 2.5× No

At each simulation step, the get_weather_impact function checks whether a given airport is under active weather at the current time. When multiple events overlap, the most restrictive capacity factor and the highest delay factor are applied. Ground stop events take precedence and halt all departures until the event ends, while arriving aircraft receive a randomized hold delay between 20 and 60 minutes.

5.4 Airport Queue Manager

The queue manager enforces capacity constraints at each airport by computing minimum separation times between consecutive operations. When a flight requests a departure or arrival slot, the manager calculates the effective capacity (base capacity multiplied by the current weather capacity factor) and determines the minimum time gap between operations.

The granted time is the later of the requested time and the earliest available slot (last operation time plus minimum separation). Any difference between the granted and requested times is recorded as queue delay, representing congestion-induced waiting. During ground stop events, departures are deferred to the event's end time, while arrivals receive stochastic holding delays.

Airport state is maintained in a mutable environment object to enable in-place updates as flights are processed sequentially. This design avoids the overhead of copying large data frames at each step while preserving R's functional idioms elsewhere in the codebase.

5.5 Flight State Machine

5.6 Core Simulation Engine

5.7 Passenger Connection Analysis

5.8 Monte Carlo Framework

5.9 Validation Framework

5.10 Sensitivity Analysis

5.11 Visualization Functions

6 Running the Simulation

6.1 Key Improvements Over Original Simulation

7 References